Code Management

Code Management — Interview Questions

Questions: 4 **Difficulty:** Mid **Companies:** Google, Amazon, Flipkart, Razorpay, Swiggy, Microsoft, Atlassian

---

Q 13.1 What are the trade-offs between monorepos and polyrepos? How do you decide?

Difficulty: Mid **Asked at:** Google, Amazon, Flipkart, Razorpay

Monorepo: All projects in one repository — frontend, backend, shared libraries, design system, configs.

Polyrepo: Each project in its own repository — `frontend-web`, `frontend-admin`, `design-system`, `api-server`.

When monorepo works well:

- Teams share code frequently (design system used by 3 apps, utility library used by everything)
- You need atomic changes across multiple packages (API contract change + frontend update in one PR)
- You want consistent tooling (one ESLint config, one CI pipeline, one set of dev dependencies)
- Your organization has 5-50 engineers working across related projects

```
monorepo/
  ├── apps/
  │   ├── web/              ← Consumer-facing site
  │   ├── admin/            ← Internal dashboard
  │   └── docs/             ← Documentation site
  ├── packages/
  │   ├── ui/               ← Shared component library
  │   ├── utils/            ← Shared utilities
  │   ├── api-client/       ← Generated API types + client
  │   └── eslint-config/    ← Shared linting rules
  └── turbo.json
```

A change to `packages/ui` automatically affects all three apps — tests run, types check, no version mismatch.

When polyrepo works well:

- Teams are fully independent (different tech stacks, different deploy schedules, no shared code)
- Strong organizational boundaries (team A shouldn't see team B's code for security/compliance)
- Very different lifecycles (a stable marketing site vs a rapidly iterating product)
- The repo would be enormous (100+ engineers — git operations slow down)

The practical decision:

If your shared code changes and you need to update consumers immediately → monorepo. If packages are versioned and consumers can adopt updates on their own schedule → polyrepo with published packages (npm).

Most mid-size frontend teams (5-30 engineers) benefit from a monorepo. The shared design system alone justifies it — publishing to npm, managing versions, and waiting for each app to upgrade creates weeks of coordination that a monorepo eliminates.

Follow-up: "What problems have you encountered with monorepos?"

harder (new engineer sees 50 packages — solved with clear README and CODEOWNERS). These are real but solvable. The coordination overhead of polyrepos is usually worse.

Q 13.2 Compare trunk-based development vs Gitflow. What role do feature flags play?

Difficulty: Mid **Asked at:** Google, Amazon, Flipkart, Razorpay, Swiggy

Gitflow — separate branches for features, releases, and hotfixes:

JAVASCRIPT Copy

```
main ──────────────────────────────────────────────────────→
     \                                          /
      develop ──────────────────────────────────
         \              /       \          /
          feature/cart──        feature/search──
                                      \    /
                                   release/2.1──
```

Every feature gets a branch. When ready, it merges to `develop`. When releasing, a `release` branch is cut from `develop`, stabilized, then merged to `main`. Hotfixes branch from `main`.

Trunk-based development — everyone commits to main (or merges short-lived branches):

JAVASCRIPT Copy

```
main ──●──●──●──●──●──●──●──●──●──●──●──●──●──→
         |     |        |
         └─┐   └─┐      └─┐
           |     |        |
        feature feature feature
```

Branches live at most 1-2 days. They're small, focused, and merged frequently. If a feature takes a week, it's merged incomplete behind a feature flag.

Why trunk-based is winning in 2026:

- **Long-lived branches cause merge hell.** A feature branch that lives for 3 weeks drifts far from `main`. The merge conflict resolution takes days and introduces bugs.
- **CI is only useful on `main`.** If your feature branch doesn't get integrated and tested against other changes for weeks, you're not really doing continuous integration.
- **Deployment frequency matters.** Companies that deploy daily or multiple times daily can't wait for 3-week branches to merge through develop → release → main.

Feature flags — the enabler:

Feature flags let you merge incomplete or experimental features to `main` without exposing them to users:

JAVASCRIPT Copy

```javascript
// Incomplete feature merged to main — hidden behind a flag
function ProductPage({ product }) {
  const showNewReviews = useFeatureFlag('new-review-system');

  return (
    <main>
      <ProductDetails product={product} />
      {showNewReviews ? (
        <NewReviewSystem productId={product.id} />
      ) : (
        <LegacyReviews productId={product.id} />
      )}
    </main>
  );
}
```

then 100%. No big-bang release. No separate branch maintenance.

Feature flag services: LaunchDarkly, Unleash, Flagsmith, or a simple in-house solution (JSON config + API endpoint).

When to still use Gitflow: Open-source projects where maintaining multiple versions is necessary (v2.x and v3.x simultaneously), or teams with formal release processes that require staging branches (regulated industries, mobile app store submissions).

Follow-up: "How do you handle feature flags that never get cleaned up?"

This is a real problem — codebases accumulate dead feature flags. Solutions: set an expiration date on every flag, run a quarterly flag cleanup, add a CI check that warns when a flag is older than 90 days, and make flag removal part of the definition of done for the feature.

Q 13.3 How does CI/CD work for frontend? What goes in the pipeline?

Difficulty: Mid **Asked at:** Every company

CI (Continuous Integration) automatically runs checks on every code change. CD (Continuous Deployment) automatically deploys passing changes to production. For frontend, the pipeline is specific to web concerns — types, tests, bundle size, performance, and preview deployments.

A production frontend CI/CD pipeline:

JAVASCRIPT Copy

```
┌─ On every Pull Request ──────────────────────────────────┐
  │                                                        │
  │  Step 1: Install dependencies                          │
  │    npm ci --frozen-lockfile    (2-3 min, cached)       │
  │                                                        │
  │  Step 2: Static analysis       (30 sec)                │
```

```
  TypeScript type check: tsc --noEmit
  Format check: prettier --check src/

Step 3: Unit + Integration tests  (1-3 min)
  vitest run --coverage
  Coverage gate: fail if < 80%

Step 4: Build                     (1-2 min)
  npm run build

Step 5: Bundle size check         (10 sec)
  bundlesize: fail if JS > 200KB

Step 6: Preview deployment        (1-2 min)
  Deploy to unique URL: pr-123.preview.example.com
  Post preview URL as PR comment
```

```
┌─ On merge to main ──────────────────────────────────────┐
│                                                         │
│  Steps 1-4: Same as above                               │
│                                                         │
│  Step 5: Lighthouse CI            (2-5 min)             │
│    Performance score > 0.9                              │
│    LCP < 2500ms                                         │
│    CLS < 0.1                                            │
│                                                         │
│  Step 6: E2E tests on critical paths  (3-5 min)         │
│    Playwright: checkout, signup, search                 │
│                                                         │
│  Step 7: Production deployment  (1-2 min)               │
│    Deploy to CDN                                        │
│    Invalidate cache                                     │
│    Upload source maps to Sentry                         │
│                                                         │
│  Step 8: Post-deploy checks       (1 min)               │
│    Smoke test: hit production URL, check for 200        │
│    Monitor error rate for 10 minutes                    │
│    Auto-rollback if error rate spikes                   │
│                                                         │
└─────────────────────────────────────────────────────────┘
```

GitHub Actions example:

YAML

Copy

```yaml
name: CI
on: [pull_request]

jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: 'npm' }
      - run: npm ci

      - name: Type check
        run: npx tsc --noEmit

      - name: Lint
        run: npx eslint src/

      - name: Test
        run: npx vitest run --coverage

      - name: Build
        run: npm run build

      - name: Bundle size
        run: npx bundlesize
```

Preview deployments — the game changer:

Every PR gets its own deployed URL (Vercel, Netlify, Cloudflare Pages do this automatically). Reviewers click the link and test the actual built application, not just read

This single feature has changed how frontend teams work more than any CI improvement in the past decade.

Follow-up: "How do you handle database migrations or API changes that the frontend depends on?"

Deploy backend changes first (backward-compatible). The new API endpoint exists but the frontend hasn't changed yet. Then deploy the frontend that uses the new endpoint. This is why backward-compatible API design matters — the frontend and backend deploy independently, and there's always a window where one is ahead of the other.

Q 13.4 What code quality tools should every frontend project have?

Difficulty: Mid **Asked at:** Amazon, Flipkart, Razorpay, Atlassian

The goal is to automate everything that can be automated so code reviews focus on logic, architecture, and UX — not formatting, import order, or missing semicolons.

ESLint — enforce code patterns and catch bugs:

JAVASCRIPT Copy

```
// .eslintrc.js
module.exports = {
  extends: [
    'eslint:recommended',
    'plugin:react/recommended',
    'plugin:react-hooks/recommended',  // Catches hook rule violations
    'plugin:@typescript-eslint/recommended',
    'plugin:jsx-a11y/recommended',     // Accessibility rules
  ],
  rules: {
    'no-console': 'warn',              // Catch forgotten console.logs
    'react/self-closing-comp': 'error', // <Component /> not <Component></
    'no-restricted-imports': ['error', {
```

```
      }],
    }],
  },
};
```

Prettier — format code automatically:

JSON Copy

```
// .prettierrc
{
  "semi": true,
  "singleQuote": true,
  "trailingComma": "all",
  "printWidth": 100,
  "tabWidth": 2
}
```

Prettier is opinionated by design — it removes formatting debates from code reviews entirely. The team agrees on one config, and every file is formatted identically.

Husky + lint-staged — enforce before commit:

JSON Copy

```
// package.json
{
  "lint-staged": {
    "*.{ts,tsx}": ["eslint --fix", "prettier --write"],
    "*.{css,json,md}": ["prettier --write"]
  }
}
```

BASH Copy

```
npx husky init
echo "npx lint-staged" > .husky/pre-commit
```

When a developer runs `git commit`, Husky triggers lint-staged which runs ESLint and Prettier only on the staged files. If ESLint fails, the commit is blocked. This prevents broken code from ever entering the repository.

Commitlint — enforce commit message format:

BASH Copy

```
# .commitlintrc.json
{
  "extends": ["@commitlint/config-conventional"]
}

# Enforces: type(scope): description
# feat(cart): add quantity selector
# fix(auth): handle expired token redirect
# chore(deps): update React to 18.3.1
```

Consistent commit messages enable automatic changelog generation (conventional-changelog), semantic versioning (semantic-release), and meaningful git history that's searchable.

The complete quality stack:

JAVASCRIPT Copy

```
Tool          | When It Runs        | What It Catches
  ————————————|———————————————————|————————————————————
  TypeScript  | Editor + CI         | Type errors, missing props
  ESLint      | Editor + pre-commit | Code patterns, a11y, hooks rules
  Prettier    | Editor + pre-commit | Formatting inconsistencies
  Husky       | git commit          | Gate — blocks bad commits
  lint-staged | git commit          | Only checks changed files (fast)
  Commitlint  | git commit          | Commit message format
  CI pipeline | PR + merge          | Tests, build, bundle size, perf
```

passing all checks. Each layer reduces the noise for the next.

Follow-up: "How do you introduce these tools to an existing project without the team revolting?"

Gradually. Add Prettier first (auto-formats, no effort from developers). Then ESLint with `--fix` on pre-commit (auto-fixes most issues). Run on changed files only (lint-staged), not the entire codebase. Enable strict rules one at a time, fixing existing violations before enabling each rule. Never dump 500 ESLint errors on a team in one PR.

QUICK CHECK — no marks, just for you.

1. A design system is consumed by three apps. Whenever the design system changes, every consuming app must pick up the change immediately rather than upgrading on its own schedule. Based on the lesson's decision rule, which repository strategy fits?

- ○ Polyrepo with the design system published as a versioned npm package
- ○ Either one — the choice only affects CI build time, not how updates propagate
- ○ Polyrepo, because three consumers already means the repo is too large for a monorepo
- ○ Monorepo — shared code that consumers must adopt immediately points to a monorepo

Check answer

2. A team practices trunk-based development. An engineer starts a feature that will realistically take a week to finish. According to the lesson, what should happen?

- ○ Cut a release branch for the feature and stabilize it there before merging to main
- ○ Hold the work locally and commit only once the feature is finished and manually tested

Merge the incomplete feature to main behind a feature flag rather than keeping the branch open for a week

Check answer

3. A frontend change depends on a brand-new API endpoint that does not exist yet. What deployment order does the lesson prescribe, and why does it work?

- Deploy the backward-compatible backend change first so the endpoint exists, then deploy the frontend that uses it
- Deploy the frontend behind a feature flag first, then remove the flag before the backend ships
- Deploy the frontend first so it is cached on the CDN, then deploy the backend endpoint it calls
- Deploy both at the same moment in a single coordinated release so no window of mismatch exists

Check answer

Previous
Tooling and environment

Next
App router and rendering

LEARN MORE

ARTICLES & DOCS

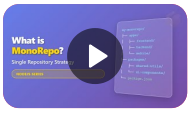
Git workflows (Atlassian)

VIDEOS



Monorepo vs multirepo

YouTube



Monorepo explained

YouTube



Git workflow & branching strategies

YouTube

Hand-picked references for this lesson.